

LLVM Optimizacije: SCCP i SimplifyCFG

Mateja Miletić 366/2022

Jovana Medenica 364/2022

SCCP - Sparse Conditional Constant Propagation

- ▶ Optimizacioni algoritam koji kombinuje dve ideje: propagaciju konstanti i analizu dostižnosti grana u kontrolnom grafu
- ▶ Otkriva vrednosti koje se mogu izračunati u vreme kompajliranja
- ▶ Otkriva delove programa koji se nikada neće izvršiti
- ▶ Cilj je da pojednostavi program i ukloni nepotrebne instrukcije
- ▶ Priprema kod za CFG

SCCP - Algoritam

- ▶ Lattice struktura: UNDEF, CONSTANT, OVERDEFINED
- ▶ CFG worklist + SSA worklist
- ▶ Praćenje izvršivih grana
- ▶ Evaluacija PHI, binarnih operacija, poređenja i cast instrukcija
- ▶ Constant folding korišćenjem LLVM

Primer 1 - konstantan uslov i mrtva grana

```
int test_sccp() {  
    int x = 2 + 3;  
  
    if (x == 5) {  
        return x + 10;  
    } else {  
        return x + 20;  
    }  
}
```

```
define dso_local i32 @test_sccp() #0 {  
entry:  
    br label %if.then  
  
if.then:                                ; preds = %entry  
    br label %return  
  
if.else:                                ; No predecessors!  
    %addl = add nsw i32 5, 20  
    br label %return  
  
return:                                ; preds = %if.else, %if.then  
    ret i32 15  
}
```

Primer 2 - ugnježdeni if uslovi

```
int test_chain_elimination() {
    int a = 6;
    int b = 4;
    int diff = a - b;
    int total = diff * 5;

    if (total > 8) {
        if (total < 15) {
            if (diff == 2) {
                return total;
            } else {
                return -3;
            }
        } else {
            return -2;
        }
    } else {
        return -1;
    }
}
```

```
define dso_local i32 @test_chain_elimination() #0 {  
entry:  
| br label %if.then  
  
if.then:                                     ; preds = %entry  
| br label %if.then2  
  
if.then2:                                   ; preds = %if.then  
| br label %if.then4  
  
if.then4:                                   ; preds = %if.then2  
| br label %return  
  
if.else:                                    ; No predecessors!  
| br label %return  
  
if.else5:                                  ; No predecessors!  
| br label %return  
  
if.else6:                                  ; No predecessors!  
| br label %return  
  
return:                                    ; preds = %if.else6, %if.  
| ret i32 10  
}
```

Primer 3 - phi čvor i izvršive grane

```
int test_phi_then_cfg() {  
    int flag = 1;  
    int val;  
  
    if (flag > 0) {  
        val = 100;  
    } else {  
        val = 200;  
    }  
  
    int result = val * 3;  
  
    if (val == 100) {  
        return result;  
    } else {  
        return 0;  
    }  
}
```

```
define dso_local i32 @test_phi_then_cfg() #0 {  
entry:  
    br label %if.then  
  
if.then:                                ; preds = %entry  
    br label %if.end  
  
if.else:                                ; No predecessors!  
    br label %if.end  
  
if.end:                                  ; preds = %if.else, %if.then  
    br label %if.then2  
  
if.then2:                                ; preds = %if.end  
    br label %return  
  
if.else3:                                ; No predecessors!  
    br label %return  
  
return:                                  ; preds = %if.else3, %if.then2  
    ret i32 300  
}
```

Implementacija SCCP Pass-a

- ▶ Implementirana kao LLVM FunctionPass
- ▶ Dve faze: analiza i primena rezultata analize
- ▶ U fazi analize se popunjavaju pomoćne strukture poput LatticeMap, ExecEdges, CFGWorklist I SSAWorklist
- ▶ U fazi analize se ne menja IR kod
- ▶ U fazi primene rezultata analize se menja IR kod: izrazi koji su dokazani da su konstante se zamenjuju konstantama, a uslovni branch čiji je uslov konstanta se zamenjuje bezuslovnim branch-om

Rezultati SCCP

- ▶ Nakon primene SCCP dobijamo jednostavnije LLVM IR oblike
- ▶ Konstantni izrazi se menjaju konstantama
- ▶ Poređenja s poznatim vrednostima postaju true ili false
- ▶ Uslovni branch gde je uslov konstanta postaju bezuslovni
- ▶ Mrtve grane se označavaju kao neizvršive
- ▶ PHI čvorovi se mogu zameniti konstantom ako svi izvršivi ulazi daju istu vrednost

SimplifyCFG

- ▶ Cilj: pojednostavljenje Control Flow Graph-a nakon prethodnih transformacija
- ▶ Bazirano na LLVM simplifycfg optimizaciji:
 - Uklanjanje nedostižnih blokova
 - Spajanje linearnih blokova
 - Eliminacija PHI čvorova
 - Preusmeravanje suvišnih bezuslovnih grana

Dead Block Elimination

- ▶ Blok koji nema prethodnike je nedostižan i može se ukloniti
- ▶ Pre uklanjanja bloka ažuriraju se PHI čvorovi u njegovim naslednicima
- ▶ Zatim se brišu instrukcije iz mrtvog bloka i uklanjaju CFG veze
- ▶ Rezultat: manji i čistiji CFG

PHI Elimination

- ▶ PHI čvor je potreban kada vrednost može doći iz više prethodnih blokova
- ▶ Ako nakon CFG čišćenja blok ima samo jednog relevantnog prethodnika, PHI postaje trivijalan
- ▶ Tada se PHI čvor zamenjuje njegovom jedinom ulaznom vrednošću
- ▶ Time se smanjuje broj SSA instrukcija i pojednostavljuje IR

Block Merging

- ▶ Ako blok ima tačno jednog prethodnika, i taj prethodnik ima baš taj blok kao jedinog naslednika, onda između njih ne postoji stvarno grananje
- ▶ Instrukcije iz oba bloka mogu se spojiti u jednu linearnu sekvencu
- ▶ Rezultat je kraći IR sa manje branch instrukcija

Redundant Branch Elimination

- ▶ Ako blok ne sadrži korisne instrukcije već samo безусловni branch, on je suvišan
- ▶ Prethodnici tog bloka mogu se preusmeriti direktno na njegov cilj i time se uklanja nepotreban skok kroz međublok
- ▶ Pri preusmeravanju se moraju ažurirati PHI čvorovi kako bi SSA semantika ostala ispravna

Primer 1 - uklanjanje mrtvog bloka

```
int test_cfg(int a) {  
    int x = a;  
  
    goto middle;  
  
middle:  
    goto exit;  
  
dead:  
    return 100;  
  
exit:  
    return x;  
}
```

```
define dso_local i32 @test_cfg(i32 noundef %a) #0 {  
entry:  
    ret i32 %a  
}
```

Primer 2 - saradnja SCCP i CFG

```
, function never, noinline, noinline, noreturn
define dso_local i32 @test_sccp() #0 {
entry:
| br label %if.then

if.then:                                ; preds = %entry
| br label %return

if.else:                                ; No predecessors!
| %add1 = add nsw i32 5, 20
| br label %return

return:                                  ; preds = %if.else, %if.then
| ret i32 15
}
```

```
define dso_local i32 @test_sccp() #0 {
entry:
| ret i32 15
}
```

Primer 3 - PHI elimination

```
define dso_local i32 @test_phi_then_cfg() #0 {
entry:
| br label %if.then

if.then:                                ; preds = %entry
| br label %if.end

if.else:                                ; No predecessors!
| br label %if.end

if.end:                                 ; preds = %if.else, %if.then
| br label %if.then2

if.then2:                               ; preds = %if.end
| br label %return

if.else3:                               ; No predecessors!
| br label %return

return:                                 ; preds = %if.else3, %if.then2
| ret i32 300
}
```

```
define dso_local i32 @test_phi_then_cfg() #0 {
entry:
| ret i32 300
}
```


Saradnja SCCP i SimplifyCFG

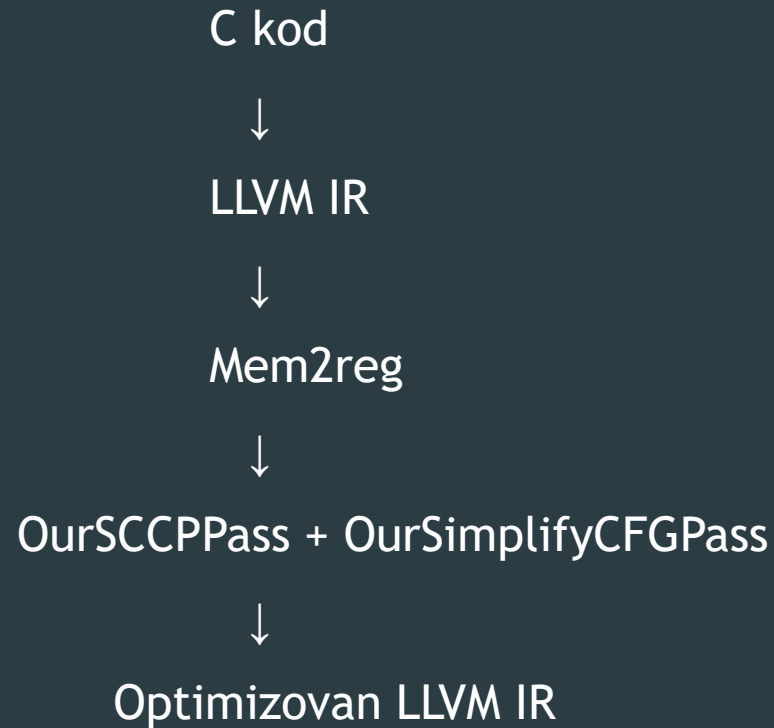
► SCCP:

- propagira konstante kroz SSA vrednosti,
- izračunava izraze čiji su operandi poznate konstante,
- pojednostavljuje branch instrukcije ako je uslov konstanta,
- neke grane označava kao neizvršive.

► SimplifyCFG:

- uklanja blokove koji su ostali bez prethodnika,
- uklanja trivijalne PHI čvorove,
- spaja linearne blokove,
- uklanja suvišne bezuslovne skokove

Arhitektura rešenja- pipeline



Zaključak

- ▶ SCCP smanjuje broj instrukcija propagacijom konstanti
- ▶ SimplifyCFG smanjuje kompleksnost CFG-a
- ▶ Kombinacija omogućava efikasniji LLVM IR
- ▶ Projekat prati LLVM SCCP i SimplifyCFG principe